Electronic Engineering Lab B

Lab 01: Your first own IP core

VHDL, simulation and AXI4-Lite IP packaging with Vivado on the Nexys 7

Prof. Dr.-Ing. Ali Hayek

22.10.2026

Goals and outcome

Goal

Design, verify and package your own AXI4-Lite IP core in VHDL.

Skills

Vivado projects, XDC constraints, XSim testbench, IP Packager.

Deliverable

Packaged pwm_axi IP, simulation screenshot, utilization and timing numbers, README.

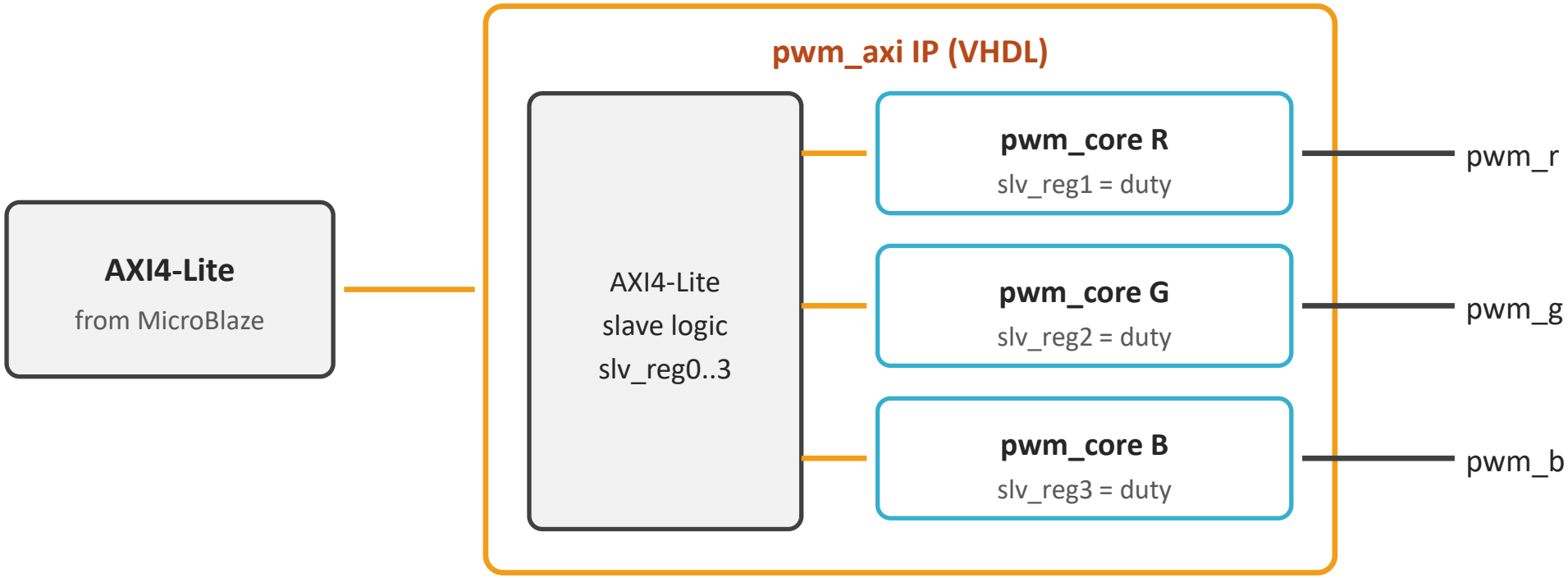
Setup checklist

Item	Check
Tools	Vivado 2026.1 or later installed; license tier must cover the Artix-7 (free Vivado BASIC tier or university license)
Board	Nexys 7 powered, USB-JTAG cable connected, power switch on
Drivers	Digilent cable drivers installed
Project folder	Short path without spaces, e.g. C:\lab\pwm
Board variant	A7-100T: xc7a100tcsg324-1

Tool versions: what to check

Item	Status or check
Release used for these slides	2025.1
Create and Package New IP	Wizard pages and option names may differ
Simulation	Check how to set the simulation top and relaunch
License tier	Must cover the Artix-7 on the board
Pin names	Use the Digilent master XDC file

What you will build



Our own work: the PWM core and its connection to the AXI4-Lite template.

Part 1 · Create the first project

1. **File, Project, New** – choose RTL Project and tick Do not specify sources at this time
2. **Name and location** – name it lab1_leds in a short path without spaces
3. **Select the part** – search for xc7a100tcsg324-1 (A7-100T) or xc7a50tcsg324-1 (A7-50T)
4. **Finish** – Vivado opens the project; set Target language to VHDL under Settings if needed
5. **Orient yourself** – find Flow Navigator, Sources, Project Summary, Tcl Console and Messages

Part 1 · First VHDL design

```
library ieee;
use ieee.std_logic_1164.all;

entity top is
  port (
    sw  : in  std_logic_vector(15 downto 0);
    led : out std_logic_vector(15 downto 0)
  );
end entity;

architecture rtl of top is
begin
  led <= sw;
end architecture;
```

Key points:

- Port names must match the XDC file
- No clock is needed: pure combinational wires
- Check Sources: top is shown as top module

Part 1 · Constraints (XDC)

```
## Excerpt: copy all 16 lines each for sw and led
## from the Digilent Nexys master XDC and rename.
set_property -dict {PACKAGE_PIN J15 IOSTANDARD LVCMOS33}
[get_ports {sw[0]}]
set_property -dict {PACKAGE_PIN L16 IOSTANDARD LVCMOS33}
[get_ports {sw[1]}]
set_property -dict {PACKAGE_PIN H17 IOSTANDARD LVCMOS33}
[get_ports {led[0]}]
set_property -dict {PACKAGE_PIN K15 IOSTANDARD LVCMOS33}
[get_ports {led[1]}]
```

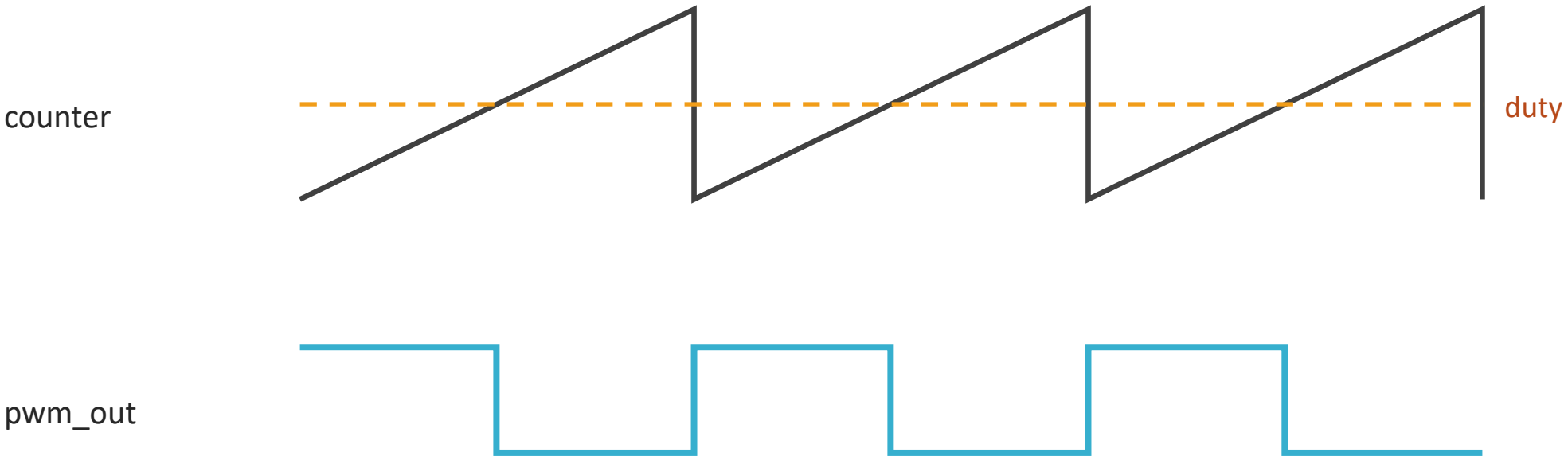
Key points:

- Use the Digilent master XDC as source
- Uncomment and rename; keep names identical to VHDL
- Verify pins against the master XDC of your board
- Unconstrained ports stop bitstream generation

Part 1 · Build and program

1. **Generate Bitstream** – Flow Navigator, runs synthesis, implementation and bitstream
2. **Read the reports** – Open Implemented Design, Report Utilization and Report Timing Summary
3. **Open Hardware Manager** – Open Target, Auto Connect; the xc7a100t device appears
4. **Program Device** – select the generated top.bit file
5. **Checkpoint** – LEDs follow the switches

Part 2 · PWM principle



Output is high while counter is below duty; one period is 2^W clock cycles

Part 2 · PWM core in VHDL

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity pwm_core is
  generic (G_WIDTH : positive := 8);
  port (clk, rst_n, enable : in  std_logic;
        duty    : in  unsigned(G_WIDTH-1 downto 0);
        pwm_out : out std_logic);
end entity;
```

```
architecture rtl of pwm_core is
  signal cnt : unsigned(G_WIDTH-1 downto 0)
    := (others => '0');
begin
  process (clk) begin
    if rising_edge(clk) then
      if rst_n = '0' then
        cnt <= (others => '0');  pwm_out <= '0';
      else
        cnt <= cnt + 1;
        if enable = '1' and cnt < duty then
          pwm_out <= '1';
        else
          pwm_out <= '0';
        end if;
      end if;
    end if;
  end process;
end architecture;
```

Generic sets the resolution · Synchronous design, active-low reset · Registered output: no glitches · Same rules as in the lecture: parameterized and standard reset

Part 2 · Self-checking testbench

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity tb_pwm is end entity;

architecture sim of tb_pwm is
  constant W : positive := 4;
  signal clk : std_logic := '0';
  signal rst_n, enable, pwm_out : std_logic := '0';
  signal duty : unsigned(W-1 downto 0)
    := to_unsigned(4, W);
begin
  clk <= not clk after 5 ns;
  dut : entity work.pwm_core generic map (G_WIDTH => W)
    port map (clk, rst_n, enable, duty, pwm_out);
  process
    variable n : natural := 0;
  begin
    wait for 20 ns; rst_n <= '1'; enable <= '1';
    wait until rising_edge(clk);
    for i in 1 to 16 loop
      wait until rising_edge(clk);
      if pwm_out = '1' then n := n + 1; end if;
    end loop;
    assert n = 4 report "Duty error" severity failure;
    report "TEST PASSED"; wait;
  end process;
end architecture;
```

Small width keeps the run short · Counts high cycles over one full period · assert makes the test self-checking · Try other duty values and a broken design

Part 2 · Run the simulation

1. **Set tb_pwm as simulation top** – Sources, Simulation Sources, right click, Set as Top
2. **Run Behavioral Simulation** – Flow Navigator, Simulation
3. **Inspect the waveform** – add cnt, set radix to unsigned decimal, zoom to fit
4. **Read the Tcl console** – look for TEST PASSED or the assertion message
5. **Experiment** – change duty to 0, 8 and 15; add a failure on purpose
6. **Checkpoint** – assertion passes for several duty values

Part 3 · Register map

Offset	Register	Bits	Function
0x00	CTRL	0	1 = enable all PWM channels
0x04	DUTY_R	7:0	Duty of the red channel
0x08	DUTY_G	7:0	Duty of the green channel
0x0C	DUTY_B	7:0	Duty of the blue channel

Part 3 · Create the AXI4 peripheral

1. **Tools, Create and Package New IP** – Next
2. **Create a new AXI4 peripheral** – name pwm_axi, version 1.0, choose an IP location
3. **Add interface** – type Lite, mode Slave, data width 32, number of registers 4
4. **Select Edit IP and Finish** – a second Vivado window opens with the IP sources
5. **Inspect the files** – top-level file and the S00_AXI file with the slv_reg0 to slv_reg3 registers

Part 3 · Add the PWM to the IP

```
-- S00_AXI entity, user ports section:
pwm_r, pwm_g, pwm_b : out std_logic;

-- S00_AXI architecture, user logic section:
u_pwm_r : entity work.pwm_core
  generic map (G_WIDTH => 8)
  port map (clk => S_AXI_ACLK, rst_n => S_AXI_ARESETN,
            enable => slv_reg0(0),
            duty => unsigned(slv_reg1(7 downto 0)),
            pwm_out => pwm_r);
-- same for green (slv_reg2, pwm_g)
-- and blue (slv_reg3, pwm_b)

-- Top-level file: add pwm_r, pwm_g, pwm_b
-- to entity, component and port map.
```

Key points:

- Three edits: S00_AXI ports, instances, top level
- Do not touch the AXI handshake code
- AXI reset is active low like our core
- Add pwm_core.vhd to the IP sources

Part 3 · Package the IP

1. **Identification** – check name, version and display name
2. **File Groups** – click Merge changes from File Groups Wizard
3. **Ports and Interfaces** – verify pwm_r, pwm_g, pwm_b and the S00_AXI interface
4. **Customization Parameters** – review the generated parameters; optional: add one for the width
5. **Review and Package** – click Package IP and close the project
6. **Checkpoint** – an IP folder with component.xml exists

Tutorials and documentation

Resource	Content	Link
AMD UG1119: Creating and Packaging Custom IP (tutorial)	Step-by-step labs with the Create and Package IP wizard; packaging projects and directories; packaging with IP Integrator	docs.amd.com/r/en-US/ug1119-vivado-creating-packaging-ip-tutorial
AMD UG1118: Creating and Packaging Custom IP (user guide)	Reference for the IP Packager: IP-XACT, file groups, parameters, inference rules	Search "UG1118" on docs.amd.com
Digilent: Getting Started with Vivado	Project, HDL sources, constraints, bitstream and programming on Digilent boards	digilent.com/reference/programmable-logic/guides/getting-started-with-vivado
Digilent: Getting Started with VHDL	VHDL introduction for Digilent boards	digilent.com/reference/programmable-logic/guides/getting-started-with-vhdl
Digilent: Nexys A7 start page and reference manual	Pinout, board I/O, master XDC files, demos	digilent.com/reference/programmable-logic/nexys-a7/start
Digilent: Getting Started with IP Integrator	Block designs with IP; useful for Labs 2 and 3	digilent.com/reference/programmable-logic/guides/getting-started-with-ipi

Which resource helps where

Lab part	Task	Where to look
Part 1	Project, VHDL top, constraints, bitstream	Digilent Getting Started with Vivado; master XDC from the Nexys A7 page
Part 2	VHDL and testbench	Digilent Getting Started with VHDL; Vivado simulation help
Part 3	Create and package an AXI4-Lite IP	AMD UG1119 tutorial and UG1118 user guide
Labs 2 and 3	Block designs with MicroBlaze and IP	Digilent Getting Started with IP Integrator
Any part	Pins, clock, board I/O, power	Nexys A7 reference manual

Create and Package New IP: quick reference

Wizard choices

Create a new AXI4 peripheral; interface type Lite, mode Slave, 32 bits, 4 registers; then Edit IP.

What you get

An IP folder, typically with component.xml, HDL and GUI files; AXI template with slv_reg0 to slv_reg3.

Search tips

Search for "Create and Package New IP", "AXI4-Lite peripheral", "UG1118", "UG1119"; use the docs of your Vivado release.

Summary and next steps

What you did

Created a PWM core, verified it in simulation and packaged it as an AXI4-Lite IP.

What you have

A reusable pwm_axi IP folder with documented registers.

Next

Lab 2: build the MicroBlaze system.
Lab 3: combine both and write software.

Thank you for your attention!

Prof. Dr. -Ing. Ali Hayek

University of Applied Sciences Hamm-Lippstadt (HSHL)

Tel.: **+49 (2381) 8789 – 924**

E-Mail.: **ali.hayek@hshl.de**